LAPORAN PRAKTIKUM

POSTTEST 5

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Nasha Nuron Wahdah (2509106037)

Kelas (A2 ‘25)

PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2025

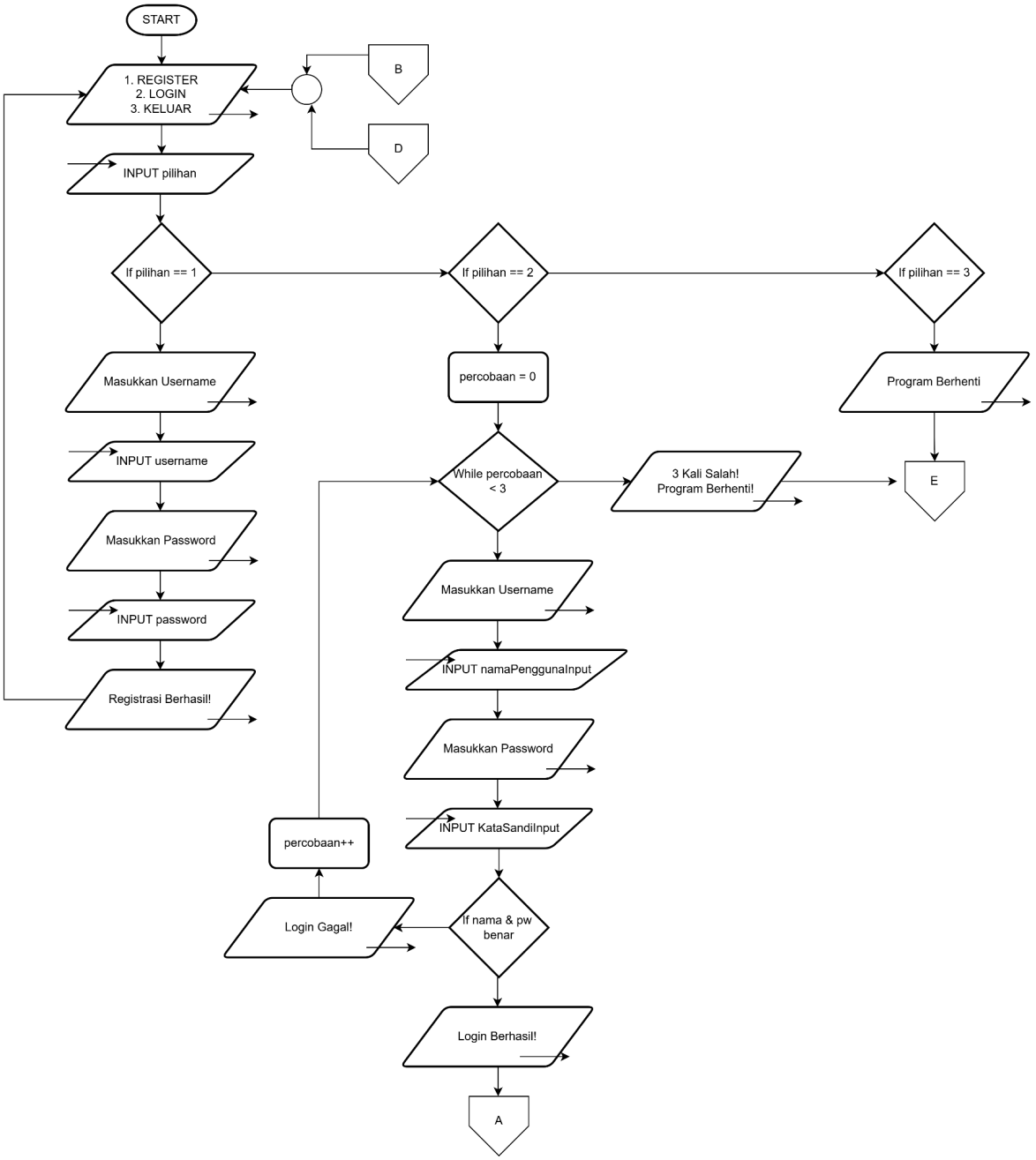
1. Flowchart

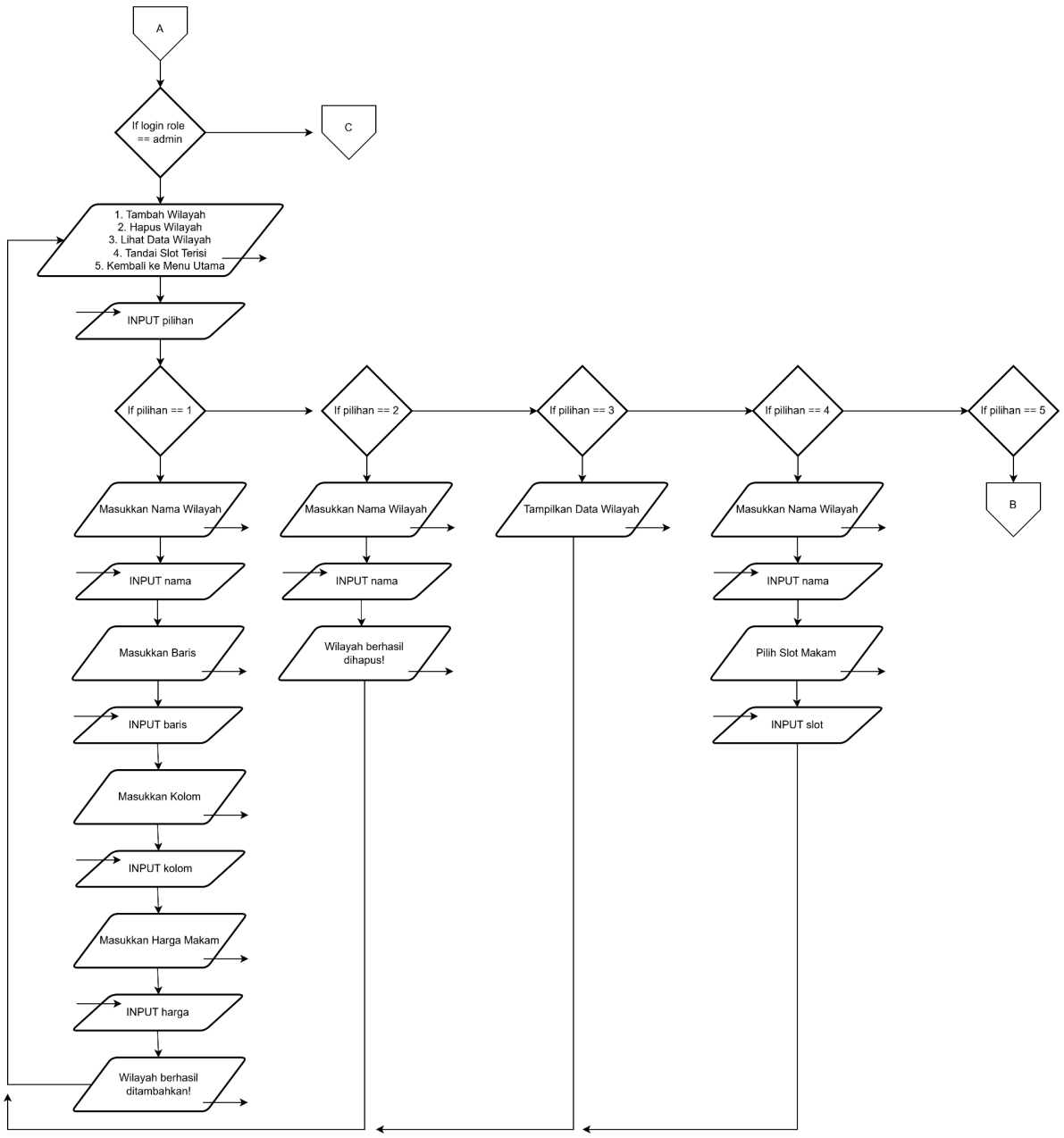
Flowchart ini menggambarkan alur program manajemen data pemakaman yang dimulai dari menu awal yang menyediakan tiga pilihan, yaitu register, login, dan keluar. Jika pengguna memilih register, pengguna diminta memasukkan username dan password untuk membuat akun baru dengan role user. Jika memilih login, pengguna harus memasukkan username dan password dengan maksimal tiga kali percobaan. Jika login gagal tiga kali, program akan berhenti. Jika login berhasil, sistem akan mengecek peran pengguna apakah sebagai admin atau user.

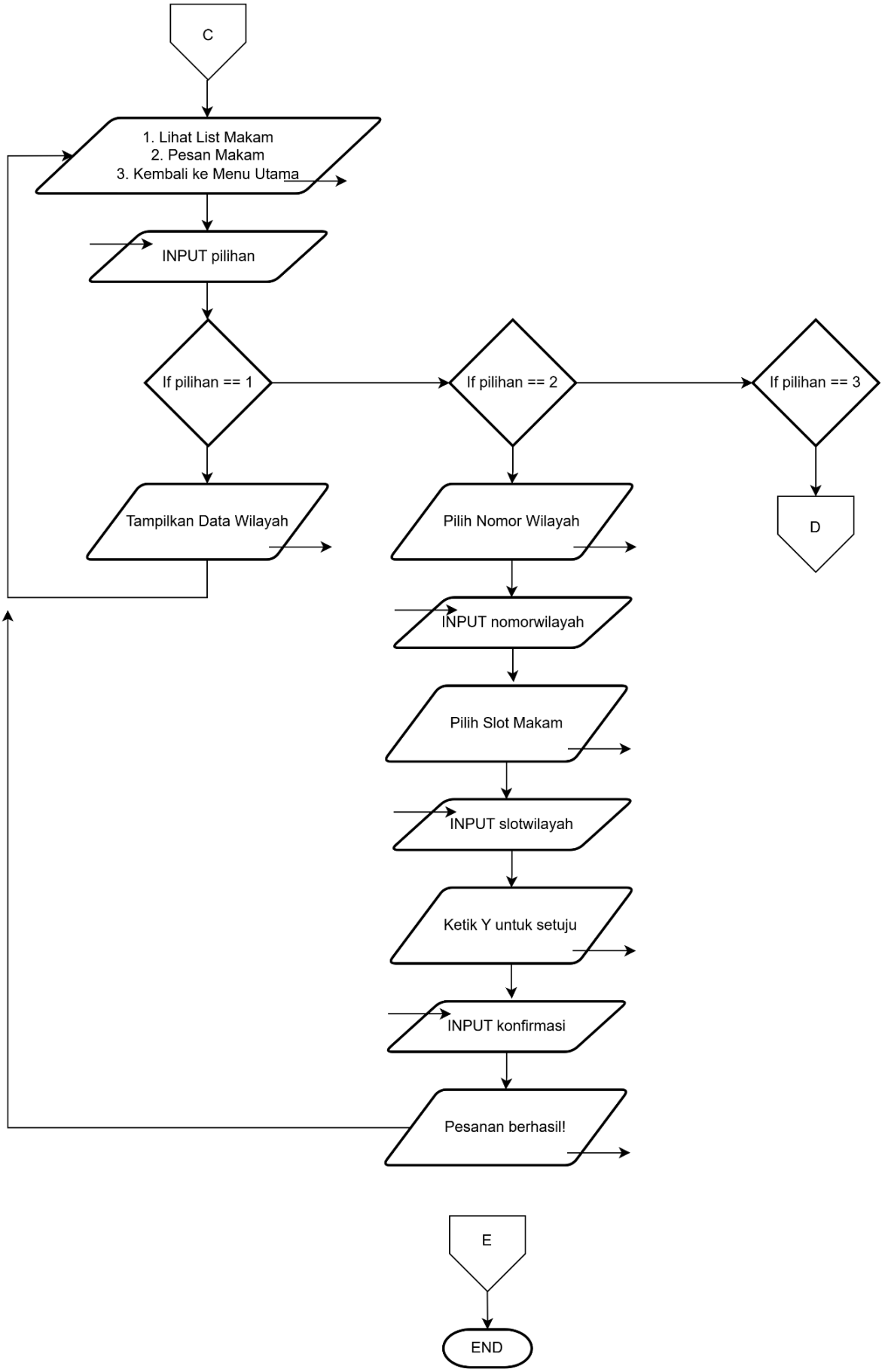
Jika yang login adalah admin, maka pengguna masuk ke menu admin yang berisi beberapa fitur pengelolaan data pemakaman, yaitu menambah wilayah pemakaman, menghapus wilayah, melihat data wilayah beserta denah slot makam, serta menandai slot makam yang sudah terisi. Pada saat menambah wilayah, admin memasukkan nama wilayah, jumlah baris dan kolom denah, serta harga per makam. Program kemudian membuat slot-slot makam sesuai kapasitas wilayah tersebut. Semua data wilayah dan slot dapat dilihat dalam bentuk daftar dan denah.

Jika yang login adalah user, maka pengguna masuk ke menu user yang menyediakan dua pilihan utama, yaitu melihat daftar makam dan memesan makam. Pada fitur melihat daftar makam, program akan menampilkan informasi wilayah pemakaman beserta denah slot makam yang menunjukkan slot yang masih tersedia dan yang sudah terisi. Jika memilih memesan makam, pengguna memilih wilayah terlebih dahulu, kemudian memilih slot yang masih tersedia. Setelah itu pengguna melakukan konfirmasi pembayaran, lalu memasukkan data jenazah seperti nama jenazah, nama orang tua, serta hari, tanggal, bulan, dan tahun wafat. Jika data yang dimasukkan valid, slot tersebut akan ditandai sebagai terisi dan pemesanan berhasil.

Program akan terus menampilkan menu sesuai peran pengguna hingga pengguna memilih kembali ke menu utama atau keluar dari program, yang akan mengakhiri jalannya program.


START
1. REGISTER
2. LOGIN
3. KELUAR
B
D
INPUT pilihan
If pilihan == 1
If pilihan == 2
If pilihan == 3
Masukkan Username
INPUT username
Masukkan Password
INPUT password
Registrasi Berhasil!
percobaan = 0
While percobaan < 3
3 Kali Salah! Program Berhenti!
Program Berhenti
E
Masukkan Username
INPUT namaPenggunaInput
Masukkan Password
INPUT KataSandiInput
If nama & pw benar
Login Gagal!
percobaan++
Login Berhasil!
A


A
If login role == admin
C
1. Tambah Wilayah
2. Hapus Wilayah
3. Lihat Data Wilayah
4. Tandai Slot Terisi
5. Kembali ke Menu Utama
INPUT pilihan
If pilihan == 1
If pilihan == 2
If pilihan == 3
If pilihan == 4
If pilihan == 5
B
Masukkan Nama Wilayah
INPUT nama
Masukkan Baris
INPUT baris
Masukkan Kolom
INPUT kolom
Masukkan Harga Makam
INPUT harga
Wilayah berhasil ditambahkan!
Masukkan Nama Wilayah
INPUT nama
Wilayah berhasil dihapus!
Tampilkan Data Wilayah
Masukkan Nama Wilayah
INPUT nama
Pilih Slot Makam
INPUT slot


C
1. Lihat List Makam
2. Pesan Makam
3. Kembali ke Menu Utama
INPUT pilihan
If pilihan == 1
If pilihan == 2
If pilihan == 3
Tampilkan Data Wilayah
Pilih Nomor Wilayah
D
INPUT nomorwilayah
Pilih Slot Makam
INPUT slotwilayah
Ketik Y untuk setuju
INPUT konfirmasi
Pesanan berhasil!
E
END

2. Deskripsi Singkat Program

Program ini secara singkat adalah program manajemen data pemakaman dengan sistem register dan login. Pengguna dapat membuat akun terlebih dahulu dengan memasukkan username dan password. Setelah itu pengguna dapat login ke dalam sistem dengan maksimal tiga kali percobaan sebelum program berhenti jika gagal.

Jika login berhasil, sistem akan mengecek peran pengguna. Jika yang login adalah admin, maka admin dapat mengelola data wilayah pemakaman seperti menambah wilayah, menghapus wilayah, melihat data wilayah beserta denah slot makam, serta menandai slot makam yang sudah terisi. Saat menambah wilayah, admin memasukkan nama wilayah, jumlah baris dan kolom denah, serta harga per makam. Program kemudian membuat slot-slot makam sesuai kapasitas wilayah tersebut.

Jika yang login adalah user, maka user dapat melihat daftar wilayah pemakaman beserta denah slot yang menunjukkan makam yang tersedia atau sudah terisi. User juga dapat melakukan pemesanan makam dengan memilih wilayah, memilih slot yang masih kosong, melakukan konfirmasi pembayaran, lalu memasukkan data jenazah seperti nama jenazah, nama orang tua, serta tanggal wafat.

Program akan terus menampilkan menu sesuai peran pengguna hingga pengguna memilih kembali ke menu utama atau keluar dari program, yang akan mengakhiri jalannya program.

3. Source Code

```
#include <iostream>
#include <string>
using namespace std;

struct Pengguna {
    string username;
    string password;
    string role;
};

struct SlotMakam {
```

```
    int id;
    int harga;
    bool terisi;
    string namaJenazah;
    string namaOrtu;
    string tanggalWafat;
};

struct Wilayah {
    string nama;
    int baris;
    int kolom;

    int totalSlot;
    int hargaPerMakam;
    SlotMakam slot[100];
};

#define MAX_PENGGUNA 50
#define MAX_WILAYAH 50

Pengguna pengguna[MAX_PENGGUNA];
int jumlahPengguna = 0;

// AKUN ADMIN
Pengguna admin = {"Nuron", "037", "admin"};

Wilayah wilayah[MAX_WILAYAH];
int jumlahWilayah = 0;

void tambahJumlahPengguna(int &jumlah) {
    jumlah++;
    cout << "Jumlah pengguna bertambah, sekarang: " << jumlah << endl;
}

void tandaiSlotDenganPointer(SlotMakam *slot, string namaJenazah, string
namaOrtu, string tanggalWafat) {
    if (slot->terisi) {
        cout << "Slot sudah terisi sebelumnya.\n";
    } else {
        slot->terisi = true;
        slot->namaJenazah = namaJenazah;
        slot->namaOrtu = namaOrtu;
        slot->tanggalWafat = tanggalWafat;
    }
}

void inisialisasiDataDummy() {
    pengguna[0] = {"budi", "123", "user"};
    pengguna[1] = {"andi", "123", "user"};
    jumlahPengguna = 2;
```

```
    wilayah[0].nama = "Bengkuring";
    wilayah[0].baris = 3; wilayah[0].kolom = 3;
    wilayah[0].totalSlot = 9;
    wilayah[0].hargaPerMakam = 750000;
    for (int i = 0; i < 9; i++) wilayah[0].slot[i] = {i + 1, 750000, false, "",
"", ""};

    wilayah[1].nama = "Sempaja";
    wilayah[1].baris = 2; wilayah[1].kolom = 5;
    wilayah[1].totalSlot = 10;
    wilayah[1].hargaPerMakam = 1000000;
    for (int i = 0; i < 10; i++) wilayah[1].slot[i] = {i + 1, 1000000, false,
"", "", ""};

    wilayah[2].nama = "LoaJanan";
    wilayah[2].baris = 2; wilayah[2].kolom = 2;
    wilayah[2].totalSlot = 4;
    wilayah[2].hargaPerMakam = 500000;
    for (int i = 0; i < 4; i++) wilayah[2].slot[i] = {i + 1, 500000, false, "",
"", ""};

    wilayah[3].nama = "Palaran";
    wilayah[3].baris = 1; wilayah[3].kolom = 5;
    wilayah[3].totalSlot = 5;
    wilayah[3].hargaPerMakam = 300000;
    for (int i = 0; i < 5; i++) wilayah[3].slot[i] = {i + 1, 300000, false, "",
"", ""};

    jumlahWilayah = 4;

    tandaiSlotDenganPointer(&wilayah[0].slot[4], "Fulan", "Bapak Fulan", "Senin,
01-01-2024");
    tandaiSlotDenganPointer(&wilayah[1].slot[9], "Joko", "Supriadi", "Rabu,
15-05-2025");
    tandaiSlotDenganPointer(&wilayah[2].slot[0], "Ahmad", "Siti", "Jumat,
10-10-2025");
    tandaiSlotDenganPointer(&wilayah[3].slot[2], "Budi", "Slamet", "Minggu,
12-12-2026");

    cout << "[SISTEM] Data dummy berhasil dimuat! Siap digunakan.\n";
}

bool masuk(const string &namaPenggunaInput, const string &kataSandiInput, string
&peran) {
    if (namaPenggunaInput == admin.username && kataSandiInput == admin.password)
{
        peran = admin.role;
        return true;
    }
    for (int i = 0; i < jumlahPengguna; i++) {
```

```
        if (pengguna[i].username == namaPenggunaInput && pengguna[i].password ==
kataSandiInput) {
            peran = pengguna[i].role;
            return true;
        }
    }
    return false;
}

void daftarPengguna() {
    if (jumlahPengguna < MAX_PENGGUNA) {
        cout << "Masukkan username: ";
        cin >> pengguna[jumlahPengguna].username;
        cout << "Masukkan password: ";
        cin >> pengguna[jumlahPengguna].password;
        pengguna[jumlahPengguna].role = "user";
        jumlahPengguna++;
        cout << "Registrasi berhasil! Akun role: user\n";
        tambahJumlahPengguna(jumlahPengguna);
    } else {
        cout << "Kapasitas user penuh!\n";
    }
}

string masukPengguna() {
    string namaPenggunaInput, kataSandiInput, peran;
    int percobaan = 0;
    while (percobaan < 3) {
        cout << "Masukkan username: ";
        cin >> namaPenggunaInput;
        cout << "Masukkan password: ";
        cin >> kataSandiInput;
        if (masuk(namaPenggunaInput, kataSandiInput, peran)) {
            cout << "Login berhasil! Selamat datang, " << namaPenggunaInput << "
(role: " << peran << ").\n";
            return peran;
        } else {
            percobaan++;
            cout << "Login gagal! Kesempatan tersisa: " << (3 - percobaan) <<
endl;
        }
    }
    cout << "Anda sudah 3 kali salah login. Program berhenti.\n";
    return "";
}

string duaDigit(int x) {
    if (x < 10) return "0" + to_string(x);
    return to_string(x);
}
```

```
void tambahWilayah() {
    if (jumlahWilayah >= MAX_WILAYAH) {
        cout << "Kapasitas wilayah penuh!\n";
        return;
    }

    Wilayah w;
    cout << "Nama wilayah (tanpa spasi, hanya huruf): ";
    cin >> w.nama;

    bool namaOk = true;
    for (int i = 0; i < (int)w.nama.length(); i++) {
        char c = w.nama[i];
        if (!((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z'))) {
            namaOk = false;
            break;
        }
    }
    if (!namaOk) {
        cout << "Nama wilayah hanya boleh huruf (tanpa spasi). Contoh:
LoaJanan\n";
        return;
    }

    cout << "Masukkan jumlah baris (angka, >0). Contoh: 4\n";
    cout << "Catatan: baris * kolom maksimal 100.\n";
    cin >> w.baris;

    cout << "Masukkan jumlah kolom (angka, >0). Contoh: 5\n";
    cin >> w.kolom;

    if (w.baris <= 0 || w.kolom <= 0) {
        cout << "Baris dan kolom harus berupa angka lebih dari 0.\n";
        return;
    }
    int kapasitas = w.baris * w.kolom;
    if (kapasitas > 100) {
        cout << "Kapasitas baris*kolom maksimal 100. Kurangi baris atau
kolom.\n";
        return;
    }
    w.totalSlot = kapasitas;

    cout << "Harga per makam (masukkan angka bulat, mis. 500000): ";
    cin >> w.hargaPerMakam;
    if (w.hargaPerMakam <= 0) {
        cout << "Harga harus berupa angka bulat lebih dari 0.\n";
        return;
    }

    for (int i = 0; i < w.totalSlot; i++) {
```

```
        w.slot[i].id = i + 1;
        w.slot[i].harga = w.hargaPerMakam;
        w.slot[i].terisi = false;
        w.slot[i].namaJenazah = "";
        w.slot[i].namaOrtu = "";
        w.slot[i].tanggalWafat = "";
    }

    wilayah[jumlahWilayah] = w;
    jumlahWilayah++;
    cout << "Wilayah berhasil ditambahkan! Total slot = " << w.totalSlot <<
"\n";
}

void tampilkanDenahWilayah(int indeks) {
    Wilayah &w = wilayah[indeks];
    int baris = w.baris;
    int kolom = w.kolom;

    int peta[100][100];
    for (int r = 0; r < baris; r++) {
        for (int c = 0; c < kolom; c++) {
            int pos = r * kolom + c + 1;
            if (pos <= w.totalSlot) peta[r][c] = pos;
            else peta[r][c] = 0;
        }
    }

    cout << "\nDenah wilayah " << w.nama << " (X = terisi)\n";
    for (int r = 0; r < baris; r++) {
        for (int c = 0; c < kolom; c++) cout << "+----";
        cout << "+\n";

        for (int c = 0; c < kolom; c++) {
            cout << "|";
            if (peta[r][c] == 0) {
                cout << "    ";
            } else {
                int num = peta[r][c];
                if (w.slot[num-1].terisi) {
                    cout << "  X ";
                } else {
                    if (num < 10) cout << "  " << num << " ";
                    else if (num < 100) cout << " " << num << " ";
                    else cout << num << " ";
                }
            }
        }
        cout << "|\n";
    }
    for (int c = 0; c < kolom; c++) cout << "+----";
```

```
    cout << "+\n";
}

void hapusWilayah() {
    if (jumlahWilayah == 0) {
        cout << "Belum ada wilayah untuk dihapus.\n";
        return;
    }
    string namaHapus;
    cout << "Masukkan nama wilayah yang akan dihapus: ";
    cin >> namaHapus;
    int indeks = -1;
    for (int i = 0; i < jumlahWilayah; i++) {
        if (wilayah[i].nama == namaHapus) {
            indeks = i;
            break;
        }
    }
    if (indeks == -1) {
        cout << "Wilayah tidak ditemukan.\n";
        return;
    }
    for (int i = indeks; i < jumlahWilayah - 1; i++) {
        wilayah[i] = wilayah[i+1];
    }
    jumlahWilayah--;
    cout << "Wilayah \"" << namaHapus << "\" berhasil dihapus.\n";
}

void tandaiSlotTerisi() {
    if (jumlahWilayah == 0) {
        cout << "Belum ada wilayah.\n";
        return;
    }
    string nama;
    cout << "Masukkan nama wilayah: ";
    cin >> nama;
    int indeks = -1;
    for (int i = 0; i < jumlahWilayah; i++) {
        if (wilayah[i].nama == nama) {
            indeks = i;
            break;
        }
    }
    if (indeks == -1) {
        cout << "Wilayah tidak ditemukan.\n";
        return;
    }

    int idSlot;
    cout << "Masukkan nomor slot yang akan ditandai terisi (1 - " <<
```

```
wilayah[indeks].totalSlot << "): ";
    cin >> idSlot;
    if (idSlot < 1 || idSlot > wilayah[indeks].totalSlot) {
        cout << "Nomor slot tidak valid.\n";
        return;
    }
    tandaiSlotDenganPointer(&wilayah[indeks].slot[idSlot-1], "Anonim", "Ortu",
"Senin, 01-01-2026");
    cout << "Slot berhasil ditandai terisi secara manual.\n";
}

void lihatDataWilayah() {
    if (jumlahWilayah == 0) {
        cout << "Belum ada data wilayah.\n";
        return;
    }
    cout << "\n--- Daftar Wilayah ---\n";
    for (int i = 0; i < jumlahWilayah; i++) {
        Wilayah *ptrWilayah = &wilayah[i];
        int tersedia = 0;
        for (int j = 0; j < ptrWilayah->totalSlot; j++) {
            if (!ptrWilayah->slot[j].terisi) tersedia++;
        }
        cout << i+1 << ". Nama: " << ptrWilayah->nama
             << " | Baris: " << ptrWilayah->baris
             << " | Kolom: " << ptrWilayah->kolom
             << " | Slot total: " << ptrWilayah->totalSlot
             << " | Tersedia: " << tersedia
             << " | Harga per makam: " << ptrWilayah->hargaPerMakam << "\n";

        tampilkanDenahWilayah(i);
    }
}

void sortWilayahNamaAsc() {
    for (int i = 0; i < jumlahWilayah - 1; i++) {
        for (int j = 0; j < jumlahWilayah - i - 1; j++) {
            if (wilayah[j].nama > wilayah[j+1].nama) {
                Wilayah temp = wilayah[j];
                wilayah[j] = wilayah[j+1];
                wilayah[j+1] = temp;
            }
        }
    }
    cout << "\n[!] Berhasil mengurutkan wilayah berdasarkan Nama (A-Z)
menggunakan Bubble Sort.\n";
}

void sortWilayahHargaDesc() {
    for (int i = 0; i < jumlahWilayah - 1; i++) {
        int maxIdx = i;
```

```
        for (int j = i + 1; j < jumlahWilayah; j++) {
            if (wilayah[j].hargaPerMakam > wilayah[maxIdx].hargaPerMakam) {
                maxIdx = j;
            }
        }
        if (maxIdx != i) {
            Wilayah temp = wilayah[i];
            wilayah[i] = wilayah[maxIdx];
            wilayah[maxIdx] = temp;
        }
    }
    cout << "\n[!] Berhasil mengurutkan wilayah berdasarkan Harga (Termahal ke
Termurah) menggunakan Selection Sort.\n";
}

void sortWilayahSlotAsc() {
    for (int i = 1; i < jumlahWilayah; i++) {
        Wilayah key = wilayah[i];
        int j = i - 1;
        while (j >= 0 && wilayah[j].totalSlot > key.totalSlot) {
            wilayah[j + 1] = wilayah[j];
            j = j - 1;
        }
        wilayah[j + 1] = key;
    }
    cout << "\n[!] Berhasil mengurutkan wilayah berdasarkan Total Slot (Sedikit
ke Banyak) menggunakan Insertion Sort.\n";
}

void menuSortingAdmin() {
    if (jumlahWilayah < 2) {
        cout << "Data wilayah kurang dari 2, tidak perlu diurutkan.\n";
        return;
    }
    int pilihan;
    cout << "\n=== MENU SORTING WILAYAH ===\n";
    cout << "1. Urutkan berdasarkan Nama (A-Z) [Bubble Sort]\n";
    cout << "2. Urutkan berdasarkan Harga (Tertinggi ke Terendah) [Selection
Sort]\n";
    cout << "3. Urutkan berdasarkan Kapasitas Slot (Terkecil ke Terbesar)
[Insertion Sort]\n";
    cout << "Pilihan: ";
    cin >> pilihan;

    switch (pilihan) {
        case 1: sortWilayahNamaAsc(); break;
        case 2: sortWilayahHargaDesc(); break;
        case 3: sortWilayahSlotAsc(); break;
        default: cout << "Pilihan tidak valid.\n";
    }
```

```
    lihatDataWilayah();
}

void menuAdmin() {
    int pilihan = 0;
    do {
        cout << "\n=== MANAJEMEN DATA PEMAKAMAN (ADMIN) ===\n";
        cout << "1. Tambah Wilayah\n";
        cout << "2. Hapus Wilayah\n";
        cout << "3. Lihat Data Wilayah (termasuk denah)\n";
        cout << "4. Tandai Slot Terisi\n";
        cout << "5. Urutkan Data Wilayah (Sorting)\n";
        cout << "6. Kembali ke menu utama\n";
        cout << "Pilihan: ";
        cin >> pilihan;

        switch (pilihan) {
            case 1: tambahWilayah(); break;
            case 2: hapusWilayah(); break;
            case 3: lihatDataWilayah(); break;
            case 4: tandaiSlotTerisi(); break;
            case 5: menuSortingAdmin(); break;
            case 6: cout << "Kembali ke menu utama.\n"; break;
            default: cout << "Pilihan tidak valid.\n";
        }
    } while (pilihan != 6);
}

void lihatDaftarMakam() {
    if (jumlahWilayah == 0) {
        cout << "Belum ada wilayah.\n";
        return;
    }
    cout << "\n--- List Makam ---\n";
    for (int i = 0; i < jumlahWilayah; i++) {
        int tersedia = 0;
        for (int j = 0; j < wilayah[i].totalSlot; j++) {
            if (!wilayah[i].slot[j].terisi) tersedia++;
        }
        cout << i+1 << ". " << wilayah[i].nama
             << " | Baris: " << wilayah[i].baris
             << " | Kolom: " << wilayah[i].kolom
             << " | Slot total: " << wilayah[i].totalSlot
             << " | Tersedia: " << tersedia
             << " | Harga: " << wilayah[i].hargaPerMakam << "\n";

        tampilkanDenahWilayah(i);
    }
}

void pesanMakam() {
```

```
    if (jumlahWilayah == 0) {
        cout << "Belum ada wilayah untuk dipesan.\n";
        return;
    }

    while (true) {
        lihatDaftarMakam();
        cout << "Pilih nomor wilayah (0 = kembali): ";
        int nomorWilayah;
        cin >> nomorWilayah;
        if (nomorWilayah == 0) return;
        if (nomorWilayah < 1 || nomorWilayah > jumlahWilayah) {
            cout << "Pilihan wilayah tidak valid.\n";
            continue;
        }
        int indeksWilayah = nomorWilayah - 1;

        tampilkanDenahWilayah(indeksWilayah);

        cout << "Slot tersedia di wilayah " << wilayah[indeksWilayah].nama <<
":\n";
        bool adaSlot = false;
        for (int i = 0; i < wilayah[indeksWilayah].totalSlot; i++) {
            if (!wilayah[indeksWilayah].slot[i].terisi) {
                cout << wilayah[indeksWilayah].slot[i].id << " ";
                adaSlot = true;
            }
        }
        cout << "\n";
        if (!adaSlot) {
            cout << "Maaf, tidak ada slot tersedia di wilayah ini.\n";
            continue;
        }

        cout << "Pilih nomor slot (0 = kembali ke pilih wilayah): ";
        int nomorSlot;
        cin >> nomorSlot;
        if (nomorSlot == 0) continue;
        if (nomorSlot < 1 || nomorSlot > wilayah[indeksWilayah].totalSlot) {
            cout << "Nomor slot tidak valid.\n";
            continue;
        }
        if (wilayah[indeksWilayah].slot[nomorSlot-1].terisi) {
            cout << "Slot sudah terisi. Pilih slot lain.\n";
            continue;
        }

        cout << "Harga untuk slot " << nomorSlot << " = " <<
wilayah[indeksWilayah].hargaPerMakam << "\n";
        cout << "Ketik 'Y' untuk bayar dan lanjut, '0' untuk kembali: ";
        string konfirmasiPembayaran;
```

```
        cin >> konfirmasiPembayaran;
        if (konfirmasiPembayaran == "0") continue;
        if (!(konfirmasiPembayaran == "Y" || konfirmasiPembayaran == "y")) {
            cout << "Pembayaran dibatalkan. Kembali ke pilih wilayah.\n";
            continue;
        }

        string namaJenazah, namaOrtu;
        cout << "Masukkan nama jenazah (satu kata, tanpa spasi): ";
        cin >> namaJenazah;
        cout << "Masukkan nama orang tua jenazah (satu kata, tanpa spasi): ";
        cin >> namaOrtu;

        string hari;
        int tanggal, bulan, tahun;
        cout << "Masukkan hari wafat (satu kata, mis. Senin): ";
        cin >> hari;
        cout << "Masukkan tanggal (1-31): ";
        cin >> tanggal;
        cout << "Masukkan bulan (1-12). Gunakan angka, bukan nama bulan. Contoh:
Januari=1, Februari=2\n";
        cin >> bulan;
        cout << "Masukkan tahun (mis. 2026): ";
        cin >> tahun;

        if (tanggal < 1 || tanggal > 31) { cout << "Tanggal tidak valid.\n";
continue; }
        if (bulan < 1 || bulan > 12) { cout << "Bulan tidak valid.\n"; continue;
}
        if (tahun <= 0) { cout << "Tahun tidak valid.\n"; continue; }

        string tanggalWafat = hari + ", " + duaDigit(tanggal) + "-" +
duaDigit(bulan) + "-" + to_string(tahun);

        tandaiSlotDenganPointer(&wilayah[indeksWilayah].slot[nomorSlot-1],
namaJenazah, namaOrtu, tanggalWafat);

        cout << "\nPemesanan berhasil! Slot " << nomorSlot << " di wilayah " <<
wilayah[indeksWilayah].nama << " telah dipesan.\n";
        return;
    }
}

void menuUser() {
    int pilihan = 0;
    do {
        cout << "\n=== MENU USER ===\n";
        cout << "1. Lihat List Makam\n";
        cout << "2. Pesan Makam\n";
        cout << "3. Kembali ke menu utama\n";
        cout << "Pilihan: ";
```

```
        cin >> pilihan;

        switch (pilihan) {
            case 1: lihatDaftarMakam(); break;
            case 2: pesanMakam(); break;
            case 3: cout << "Kembali ke menu utama.\n"; break;
            default: cout << "Pilihan tidak valid.\n";
        }
    } while (pilihan != 3);
}

int main() {
    inisialisasiDataDummy();

    int pilihan;
    do {
        cout << "\n=== MENU REGISTER & LOGIN ===\n";
        cout << "1. Register\n";
        cout << "2. Login\n";
        cout << "3. Keluar\n";
        cout << "Pilihan: ";
        cin >> pilihan;
        switch (pilihan) {
            case 1: daftarPengguna(); break;
            case 2: {
                string peran = masukPengguna();
                if (peran == "") return 0;
                if (peran == "admin") {
                    menuAdmin();
                } else {
                    menuUser();
                }
                break;
            }
            case 3: cout << "Program selesai.\n"; break;
            default: cout << "Pilihan tidak valid.\n";
        }
    } while (pilihan != 3);
    return 0;
}
```

4. Langkah-langkah GIT

4.1 GIT Init

GIT Init berfungsi untuk membuat repository Git baru di folder yang sedang aktif. Dengan perintah ini, direktori tersebut mulai bisa dipakai untuk menyimpan dan mengelola versi kode menggunakan Git.

```
PS D:\KULIAH\APL\Praktikum\GIT> git init
Initialized empty Git repository in D:/KULIAH/APL/Praktikum/GIT/.git/
```

4.2 GIT Add

GIT Add dipakai untuk menandai file yang sudah diubah supaya siap dimasukkan ke commit. Jadi intinya, perintah ini menyiapkan file untuk disimpan ke riwayat Git.

```
PS D:\KULIAH\APL\Praktikum\GIT> git add .
```

4.3 GIT Commit

GIT Commit digunakan untuk menyimpan perubahan yang sudah ditambahkan ke staging area ke dalam riwayat repository Git. Perintah ini membuat catatan baru berisi perubahan kode beserta pesan yang menjelaskan perubahan tersebut.

```
PS D:\KULIAH\APL\Praktikum\GIT> git commit -m "Finish Post Test 1"
[main 4a1ea22] Finish Post Test 1
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 praktikum-apl/post-test/post-test-apl-1/Template Laporan APL.docx
```

4.4 GIT Remote

GIT Remote digunakan untuk mengatur alamat repository jarak jauh. Dengan perintah ini, repository lokal bisa dihubungkan ke repository di server seperti GitHub untuk melakukan push dan pull kode.

```
PS D:\KULIAH\APL\Praktikum\GIT> git remote add origin https://github.com/nashanuronwahdah/praktikum-apl.git
```

4.5 GIT Push

GIT Push digunakan untuk mengirim commit dari repository lokal ke repository jarak jauh. Dengan perintah ini, perubahan yang sudah disimpan akan diupload ke server seperti GitHub.

```
PS D:\KULIAH\APL\Praktikum\GIT> git push origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 273.04 KiB | 18.20 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/nashanuronwahdah/praktikum-apl.git
   6a6d20b..4a1ea22  main -> main
```